

Antigravity Skills for Optimal Development

https://gemini.google.com/app/84aba1f69c1e25e4

User prompt: Proposes moi une liste de SKILLs dont mon compte Antigravity aurait besoin pour bien fonctionner niveau identification des failles de sécurité, de bon langages de codes frontend UI/UX, de Base de données Supabase et de backend pour tout mes projets afin que les agents puissent exécuter leurs tâches avec une précision d'expertise optimisé afin d'avoir de très bon résultats à chaque développement que ce soit en applications, Saas, Sites Web, etc... Fais moi la liste des meilleurs et des plus sophistiqués pour un rendu très optimal.

Response: Pour qu'un agent **Google Antigravity** opère avec la précision d'un ingénieur senior, il doit disposer de compétences (*Skills*) sous forme de modules **SKILL.md**. Dans Antigravity, un Skill combine des consignes architecturales, des arbres de décision et des scripts d'exécution automatique (via Python, Bash ou serveurs MCP).

Voici la sélection des **SKILLs les plus sophistiqués** à déployer dans votre dossier global (~/.gemini/config/skills/) ou dans chaque projet (.agents/skills/) pour garantir un niveau d'excellence sur l'ensemble de votre stack (SaaS, Apps Web & Mobile).

1. Sécurité & Identification des Failles

Nom du SKILL	Domaine d'Expertise	Rôle & Actions de l'Agent
security-vulnerability-scanner	OWASP Top 10 & SAST/DAST	Détecte les failles XSS, SQLi, CSRF, SSRF, et analyse statique des dépendances (npm audit, pip audit).
auth-rbac-guard	Authentification & Autorisations	Inspecte les flux OAuth, JWT, la gestion de sessions, le hachage de mots de passe et le contrôle d'accès (RBAC/ABAC).
secrets-leak-detector	Prévention des fuites	Scanne le code et l'historique Git à la recherche d'API Keys, clés privées ou variables d'environnement exposées en dur.

Rendu attendu : L'agent effectue un audit de sécurité automatisé avant tout commit ou merge, générant un rapport de vulnérabilité avec correctifs immédiats.

2. Frontend UI/UX & Design Systems

Nom du SKILL	Domaine d'Expertise	Rôle & Actions de l'Agent
senior-frontend-architect	Next.js / React / Vue / Flutter	Architecture des composants, Server Components (RSC), gestion d'état globale, hydratation et performances Web Vitals.
ui-design-system-builder	Tailwind CSS / Shadcn UI	Génération et maintien de tokens de design, thématisation (Dark/Light mode), layout responsive et composants réutilisables.
ux-accessibility-wcag	Accessibilité & Audit UX	Audit de conformité WCAG 2.2 AA, gestion du focus au clavier, attributs ARIA et optimisation des animations fluides.

Rendu attendu : Des interfaces modernes, ultra-rapides, adaptées aux écrans mobiles/desktop, prêtes pour la production sans dette technique UI.

3. Base de Données Supabase & PostgreSQL

Nom du SKILL	Domaine d'Expertise	Rôle & Actions de l'Agent
supabase-master-architect	PostgreSQL & Schema Design	Conception du schéma de données, modélisation relationnelle, indexation avancée et requêtes SQL optimisées.
supabase-rls-policy-engine	Row Level Security (RLS)	Écriture et vérification des règles de sécurité au niveau des lignes (RLS) pour isoler strictement les données utilisateur/multi-tenant.
supabase-edge-functions	Serverless / Deno / Webhooks	Développement de fonctions Edge sécurisées, gestion des triggers Realtime, Auth Hooks et intégration de services tiers (Stripe, etc.).
db-migration-manager	Versioning & Migrations	Gestion propre des migrations Supabase CLI, schémas de rollback et scripts de données de test (<i>seeders</i>).

4. Backend & API Architecture

Nom du SKILL	Domaine d'Expertise	Rôle & Actions de l'Agent
backend-api-architect	Node.js / TypeScript / FastAPI	Modélisation d'API RESTful & GraphQL, validation stricte des entrées (Zod/Pydantic) et gestion centralisée des erreurs.
resilient-backend-patterns	Caching, Queues & Scaling	Mise en place de cache Redis, files de traitement asynchrones (BullMQ/Celery), rate-limiting et idempotence des requêtes.

5. Assurance Qualité & Revue de Code (Qualité Globale)

Nom du SKILL	Domaine d'Expertise	Rôle & Actions de l'Agent
senior-code-reviewer	Code Quality & Refactoring	Analyse de la lisibilité, du découpage modulaire, du DRY (Don't Repeat Yourself) et de la maintenabilité du code.
autonomous-test-suite	Vitest / Playwright / Jest	Génération autonome de tests unitaires, d'intégration et End-to-End (E2E), avec auto-correction en cas d'échec.

Modèle d'Architecture d'un SKILL Antigravity

Pour installer l'un de ces skills dans votre environnement Antigravity, vous créez un dossier dans `.agents/skills/<nom-du-skill>/` avec le fichier `SKILL.md`.

Voici la structure recommandée pour le SKILL `supabase-rls-policy-engine` :

```
---
name: supabase-rls-policy-engine
description: Expert en conception et audit de politiques RLS (Row Level Security) sur Supabase PostgreSQL. À utiliser
---

# Supabase Row Level Security (RLS) Protocol

## Consignes Générales
1. Toute nouvelle table PostgreSQL DOIT avoir RLS activé immédiatement :
  `ALTER TABLE table_name ENABLE ROW LEVEL SECURITY;`
2. Ne JAMAIS laisser de table sans politique d'accès explicite.
3. Séparer systématiquement les politiques pour `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

## Arbre de Décision
- **Données publiques en lecture seule** : Autoriser `SELECT` pour `anon` et `authenticated`.
- **Données propres à l'utilisateur** : Vérifier `auth.uid() = user_id`.
- **Multi-tenant / Équipes** : Vérifier l'appartenance de l'utilisateur à l'organisation via une fonction Security D

## Checklist de Vérification
- [ ] RLS est-il activé (`ENABLE ROW LEVEL SECURITY`) ?
- [ ] Les requêtes de politique évitent-elles les récursions infinies ?
- [ ] Les colonnes clés utilisées dans les politiques sont-elles indexées ?
```

Comment déployer ces SKILLS dans votre workflow Antigravity

- 1. **Compétences Générales (Globales)** : Placez les skills transversaux (`security-vulnerability-scanner`, `senior-code-reviewer`, `ui-design-system-builder`) dans votre dossier utilisateur `~/.gemini/config/skills/`.
- 2. **Compétences Spécifiques au Projet (Projet/Workspace)** : Placez les skills liés aux bases de données et architectures spécifiques (`supabase-master-architect`, `supabase-rls-policy-engine`) dans le dossier `.agents/skills/` à la racine de votre dépôt.
- 3. **Activation Automatique** : Antigravity sélectionnera automatiquement le SKILL approprié en fonction du contexte de votre demande et du type de tâche exécutée par l'agent.

Sources

- 1. [Google Antigravity 2.0 - Skills Hub](#)
- 2. [Skills - Google Antigravity Docs](#)
- 3. [Authoring Google Antigravity Skills - Codelabs](#)

User prompt: Génère moi le contenu complet du fichier `SKILL.md` pour le `security-vulnerability-scanner` prêt à être copié dans Antigravity.

Response: Voici le contenu complet et prêt à l'emploi du fichier `SKILL.md` pour la compétence `security-vulnerability-scanner`.

Le skill a également été automatiquement sauvegardé et activé dans votre environnement sous le nom `security-vulnerability-scanner`.

Contenu du fichier `SKILL.md`

```
---
name: security-vulnerability-scanner
description: Protocoles d'analyse statique et dynamique de sécurité (SAST/DAST), identification des vulnérabilités (
---

# Security Vulnerability Scanner Protocol

## Executive Summary & Mission
Ce SKILL régit les mécanismes d'audit de sécurité, de détection proactive des vulnérabilités et de renforcement des
---
```

1. Périmètre de Détection & Couverture OWASP Top 10

A. Injections (SQL, NoSQL, Command, ORM)

- **Détection :** Concaténation de requêtes SQL brutes, requêtes dynamiques non paramétrées, exécution de commandes
- **Règle absolue :** Toujours utiliser des requêtes préparées (*Prepared Statements*), des ORM sécurisés (Prisma, D

B. Access Control & Authorization Failure (IDOR, Broken RBAC)

- **Détection :**
 - Endpoints d'API sans vérification du jeton d'authentification (`Bearer Token` / `JWT`).
 - Absence de vérification des droits sur les ressources (`user\_id` en paramètre d'URL sans contrôle de propriété p
 - Tables Supabase/PostgreSQL avec RLS désactivé ou mal configuré (`policy USING (true)` inapproprié).
- **Règle absolue :** Appliquer le principe du moindre privilège (*Zero Trust*). Tout accès aux ressources doit être

C. Cryptographie & Gestion des Données Sensibles

- **Détection :**
 - Mots de passe stockés en clair ou avec des algorithmes faibles (`MD5`, `SHA1`).
 - Clés d'API, jetons JWT, chaînes de connexion DB inscrites en clair (*Hardcoded*) dans le code source.
 - Transmission HTTP non sécurisée sans HTTPS/TLS strict.
- **Règle absolue :** Hachage via `Argon2id` ou `bcrypt`. Secrets exclusivement dans des variables d'environnement (

D. Server-Side Request Forgery (SSRF) & Cross-Site Scripting (XSS)

- **Détection :**
 - Injection de HTML/JSEX non assaini (`dangerouslySetInnerHTML`, `v-html`, ou concaténation directe d'entrées utili
 - Appels `fetch()` ou `axios` vers des URLs fournies par l'utilisateur sans validation d'IP/domaine (permettant le
- **Règle absolue :** Assainissement strict avec `DOMPurify`. Validation par liste blanche (*Allowlist*) pour toutes

2. Détection Automatisée de Fuites & Audit de Dépendances

L'agent doit exécuter ou simuler les vérifications suivantes selon la stack :

NPM / Node.js

```
```bash
npm audit --json
npx gitleaks detect --source . -v
```

### Python / FastAPI

```
pip-audit
bandit -r ./ -f json
```

### Secret Leak Detection (Regex & Git History)

Scan systématique à la recherche des motifs :

- Google / Gemini API Keys : AIzaSy[A-Za-z0-9\_-]{35}
- Stripe Keys : sk\_live\_[0-9a-zA-Z]{24} / rk\_live\_[0-9a-zA-Z]{24}
- Supabase Service Role Key : eyJhbGciOi...
- Clés Privées RSA/SSH : -----BEGIN PRIVATE KEY-----

## 3. Checklist d'Audit Avant Validation de Code

Avant de finaliser une fonctionnalité ou d'effectuer un commit, l'agent doit valider la checklist suivante :

- [ ] **Validation des Entrées :** Toutes les données utilisateur (req.body, req.query, req.params) sont-elles validées par un schéma strict (ex. Zod) ?
- [ ] **Authentication & Access Control :** La route exige-t-elle une session active ? Les droits d'accès sont-ils vérifiés ?
- [ ] **Protocole CORS & CSRF :** Les origines autorisées (*Allowed Origins*) sont-elles explicitement restreintes au lieu de \* en production ?
- [ ] **En-têtes de Sécurité (HTTP Security Headers) :** Helms/Middleware configuré avec :
  - Content-Security-Policy (CSP)
  - X-Frame-Options: DENY
  - X-Content-Type-Options: nosniff
  - Strict-Transport-Security (HSTS)
- [ ] **Gestion des Erreurs :** Les exceptions retournent-elles des messages d'erreur génériques sans exposer de stack traces ou détails internes au client ?
- [ ] **Rate Limiting :** Les routes sensibles (login, register, reset-password, API IA) sont-elles protégées par un limiteur de débit (Rate Limiter / Redis) ?

## 4. Modèle de Rapport de Sécurité (Output Format)

Lorsque l'agent identifie une vulnérabilité, il doit structurer sa réponse selon le format suivant :

 **Alerte de Sécurité : [Titre de la vulnérabilité]**

- **Niveau de Sécurité** : [CRITIQUE / ÉLEVÉ / MOYEN / FAIBLE]
- **Catégorie OWASP** : [ex: A01:2021-Broken Access Control]
- **Fichier / Ligne concerné(e)** : src/api/user.ts:42

 **Code Vulnérable :**

// Exemple de code insécurisé identifié

 **Correctif Proposé (Secured Code) :**

// Code corrigé respectant les normes de sécurité

 **Impact & Explication :**

[Explication concise de la faille et comment le correctif la colmate.]

[http://googleusercontent.com/action\\_card\\_content/08a76a56-b33b-4e00-bc87-5407be4ffd19](http://googleusercontent.com/action_card_content/08a76a56-b33b-4e00-bc87-5407be4ffd19)

User prompt: Génère moi le fichier SKILL.md complet et optimisé pour le Supabase Master Architect et les règles RLS.

Response: Voici le contenu complet et optimisé du fichier SKILL.md pour le skill supabase-master-architect (incluant l'architecture globale Supabase et le moteur de règles RLS).

Le skill a également été enregistré et activé dans votre environnement Antigravity sous le nom supabase-master-architect.

### Contenu du fichier SKILL.md

```

name: supabase-master-architect
description: Architecture avancée de bases de données Supabase / PostgreSQL, règles de sécurité Row Level Security (

Supabase Master Architect & RLS Policy Protocol

Executive Summary & Mission
Ce SKILL régit la conception du schéma de données, l'implémentation de la sécurité Row Level Security (RLS), l'optim

1. Principes Fondamentaux de Sécurité RLS (Zero Trust)

A. Règle D'Or d'Activation
1. **RLS Obligatoire** : Toute table créée doit OBLIGATOIREMENT être sécurisée dès sa création :
  ```sql
  ALTER TABLE table_name ENABLE ROW LEVEL SECURITY;
  
```

2. **Deny by Default** : Par défaut, une fois RLS activé sans politique explicitée, aucun accès (SELECT, INSERT, UPDATE, DELETE) n'est accordé à quiconque (anon ou authenticated).

3. **Séparation des Intentions** : Ne JAMAIS regrouper les droits avec FOR ALL. Définir des politiques granulaires et séparées pour FOR SELECT, FOR INSERT, FOR UPDATE et FOR DELETE.

2. Patterns Recommandés pour les Politiques RLS

A. Pattern 1 : Données Personnelles Utilisateur (Single Tenant / User-owned)

```
-- Lecture : Un utilisateur ne peut lire que ses propres données
CREATE POLICY "Users can read own data"
ON profiles FOR SELECT
TO authenticated
USING (auth.uid() = user_id);

-- Insertion : L'utilisateur ne peut insérer qu'avec son propre ID
CREATE POLICY "Users can insert own data"
ON profiles FOR INSERT
TO authenticated
WITH CHECK (auth.uid() = user_id);
```

B. Pattern 2 : Multi-Tenant / Organisation / Équipe

Pour éviter les requêtes lentes lors de la vérification de l'appartenance à un groupe ou une équipe, utiliser une fonction SECURITY DEFINER dédiée et indexée.

```
-- Fonction optimisée pour vérifier l'appartenance à une organisation
CREATE OR REPLACE FUNCTION public.is_org_member(_user_id uuid, _org_id uuid)
RETURNS boolean
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
STABLE
AS $$SELECT EXISTS (      SELECT 1      FROM organization_members      WHERE user_id = _user_id AND organization_id =

-- Politique RLS utilisant la fonction
CREATE POLICY "Org members can view org projects"
ON projects FOR SELECT
TO authenticated
USING (is_org_member(auth.uid(), organization_id));
```

C. Pattern 3 : Accès Public en Lecture Seule

```
CREATE POLICY "Public read access for published posts"
ON posts FOR SELECT
TO anon, authenticated
USING (status = 'published');
```

3. Automatisation & Triggers Indispensables

A. Synchronisation Automatique de auth.users vers public.profiles

Toute création d'utilisateur via Supabase Auth doit automatiquement alimenter la table publique profiles :

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER SET search_path = public
AS $$ BEGIN  INSERT INTO public.profiles (id, full_name, avatar_url, updated_at)  VALUES (      new.id,      new.raw

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

B. Mise à Jour Automatique du Champ updated_at

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$ BEGIN  NEW.updated_at = now();  RETURN NEW; END; $$ language 'plpgsql';

CREATE TRIGGER set_updated_at
  BEFORE UPDATE ON public.my_table
  FOR EACH ROW
  EXECUTE PROCEDURE update_updated_at_column();
```

4. Performance & Indexation PostgreSQL

1. Indexation des Clefs Étrangères & Colonnes RLS :

- Toutes les colonnes référencées dans les règles RLS (user_id, organization_id, tenant_id) **DOIVENT** posséder un index B-Tree pour éviter les Sequential Scans systématiques.

```
CREATE INDEX idx_projects_user_id ON projects(user_id);
CREATE INDEX idx_projects_org_id ON projects(organization_id);
```

2. Utilisation des UUIDs :

- Utiliser gen_random_uuid() comme valeur par défaut pour les clés primaires ID.

3. Optimisation des Jointures RLS :

- Éviter d'interroger directement d'autres tables avec RLS dans une clause USING sans passer par des fonctions STABLE / SECURITY DEFINER.

5. Checklist d'Audit Supabase Avant Production

L'agent doit obligatoirement vérifier les points suivants :

- [] RLS est-il explicitement activé (ALTER TABLE ... ENABLE ROW LEVEL SECURITY) sur TOUTES les tables du schéma public ?

- [] La clef `service_role` (Service Key) est-elle STRICTEMENT réservée aux environnements Backend/Edge Functions sécurisés et JAMAIS exposée côté Frontend ?
- [] Les fonctions `SECURITY DEFINER` ont-elles une définition explicite du `search_path` (`SET search_path = public`) pour prévenir les attaques par injection de schéma ?
- [] Des index existent-ils sur toutes les colonnes testées dans la clause `USING` ou `WITH CHECK` des politiques RLS ?
- [] Les données sensibles (tokens, identifiants de paiement) sont-elles isolées dans des tables spécifiques avec politiques restreintes ?

6. Modèle de Script DDL Recommandé (Output Standard)

Lorsque l'agent génère un schéma de données pour Supabase, il doit suivre ce format standardisé :

```
-- 1. Déclaration de la table
CREATE TABLE IF NOT EXISTS public.tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  completed BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 2. Activation RLS
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;

-- 3. Index de performance
CREATE INDEX IF NOT EXISTS idx_tasks_user_id ON public.tasks(user_id);

-- 4. Politiques RLS granulaires
CREATE POLICY "Users can select own tasks"
ON public.tasks FOR SELECT TO authenticated
USING (auth.uid() = user_id);

CREATE POLICY "Users can insert own tasks"
ON public.tasks FOR INSERT TO authenticated
WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users can update own tasks"
ON public.tasks FOR UPDATE TO authenticated
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users can delete own tasks"
ON public.tasks FOR DELETE TO authenticated
USING (auth.uid() = user_id);

-- 5. Trigger updated_at
CREATE TRIGGER set_tasks_updated_at
BEFORE UPDATE ON public.tasks
FOR EACH ROW
EXECUTE PROCEDURE update_updated_at_column();

<ElicitationsGroup message="Souhaitez-vous générer les SKILLS restants ?">
  <Elicitation label="Créer le SKILL.md UI Design System Builder (Tailwind/Shadcn)" query="Génère moi le fichier SKI
  <Elicitation label="Créer le SKILL.md Senior Frontend Architect" query="Génère moi le fichier SKILL.md complet pou
  <Elicitation label="Créer le SKILL.md Backend API Architect" query="Génère moi le fichier SKILL.md complet pour le
</ElicitationsGroup>

http://googleusercontent.com/action_card_content/6a3255ed-654c-4b85-9453-1e0217ce2676
```

User prompt: Rédige le contenu du fichier SKILL.md complet pour le UI Design System Builder avec Tailwind CSS et Shadcn/ui.

Response: Voici le contenu complet et optimisé du fichier SKILL.md pour le skill ui-design-system-builder (Tailwind CSS & Shadcn/ui).

Le skill a été enregistré et activé dans votre environnement Antigravity sous le nom `ui-design-system-builder`.

Contenu du fichier SKILL.md

```
---
name: ui-design-system-builder
description: Directives d'architecture UI/UX, tokens de design, Tailwind CSS, Shadcn/ui, composants réutilisables, t
---

# UI Design System Builder Protocol (Tailwind CSS & Shadcn/ui)
```

Executive Summary & Mission

Ce SKILL régit la création d'interfaces utilisateur (UI) et d'expériences utilisateur (UX) modernes, fluides, access

1. Principes Design & Architecture Système

A. Modularité & Composants Atomiques

- **Composants atomiques** : Découper les interfaces en composants réutilisables, isolés et sans effets de bord.
- **Principe CVA (Class Variance Authority)** : Utiliser `cva` pour gérer proprement les variantes de composants (`v`)
- **Composition over Inheritance** : Préférer le pattern `Slot` (Radix UI / Shadcn) pour permettre la composition fl

B. Tokens de Design via Variables CSS (`globals.css`)

Toutes les couleurs, rayons de bordures et ombres doivent reposer sur des variables CSS HSL pour garantir le support

```css

```
@layer base {
 :root {
 --background: 0 0% 100%;
 --foreground: 222.2 84% 4.9%;
 --card: 0 0% 100%;
 --card-foreground: 222.2 84% 4.9%;
 --popover: 0 0% 100%;
 --popover-foreground: 222.2 84% 4.9%;
 --primary: 222.2 47.4% 11.2%;
 --primary-foreground: 210 40% 98%;
 --secondary: 210 40% 96.1%;
 --secondary-foreground: 222.2 47.4% 11.2%;
 --muted: 210 40% 96.1%;
 --muted-foreground: 215.4 16.3% 46.9%;
 --accent: 210 40% 96.1%;
 --accent-foreground: 222.2 47.4% 11.2%;
 --destructive: 0 84.2% 60.2%;
 --destructive-foreground: 210 40% 98%;
 --border: 214.3 31.8% 91.4%;
 --input: 214.3 31.8% 91.4%;
 --ring: 222.2 84% 4.9%;
 --radius: 0.5rem;
 }

 .dark {
 --background: 222.2 84% 4.9%;
 --foreground: 210 40% 98%;
 --card: 222.2 84% 4.9%;
 --card-foreground: 210 40% 98%;
 --popover: 222.2 84% 4.9%;
 --popover-foreground: 210 40% 98%;
 --primary: 210 40% 98%;
 --primary-foreground: 222.2 47.4% 11.2%;
 --secondary: 217.2 32.6% 17.5%;
 --secondary-foreground: 210 40% 98%;
 --muted: 217.2 32.6% 17.5%;
 --muted-foreground: 215 20.2% 65.1%;
 --accent: 217.2 32.6% 17.5%;
 --accent-foreground: 210 40% 98%;
 --destructive: 0 62.8% 30.6%;
 --destructive-foreground: 210 40% 98%;
 --border: 217.2 32.6% 17.5%;
 --input: 217.2 32.6% 17.5%;
 --ring: 212.7 26.8% 83.9%;
 }
}
```

## 2. Règles Tailwind CSS & Utility Utilities

1. **Utilisation Obligatoire de cn() (clsx + tailwind-merge)** : Toutes les classes Tailwind conditionnelles ou passées via des props className **DOIVENT** être fusionnées avec le helper cn().

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

2. **Responsive First & Layout Fluidity** :

- Toujours adopter l'approche Mobile-First (ex: grid-cols-1 md:grid-cols-2 lg:grid-cols-3).

- Préférer la cohérence du spacement Tailwind (space-y-4, gap-6, p-4, p-6) plutôt que des valeurs arbitraires comme p-[17px].

### 3. Animations et Transitions Subtiles :

- Utiliser des transitions douces : `transition-colors duration-200 ease-in-out` ou `framer-motion / tailwindcss-animate` pour les modales, drawers et menus déroulants.
- Ne pas surcharger l'interface d'animations agressives.

## 3. Accessibilité (WCAG 2.2 AA) & Ergonomie

- **Contraste des Couleurs** : Garantir un ratio de contraste minimal de 4.5:1 pour le texte normal et 3:1 pour le texte grand/titres.
- **Navigabilité au Clavier** :
  - Tous les éléments interactifs (`<button>`, `<a>`, `<input>`) doivent disposer d'un état de focus visible (`focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring`).
  - Préférer les composants Radix UI (base de Shadcn) pour la gestion automatique des touches ARIA (Escape, Tab, flèches directionnelles).
- **Semantique HTML Strict** :
  - Utiliser `<main>`, `<header>`, `<nav>`, `<footer>`, `<section>`, `<article>`, `<button>` plutôt que des `<div>` à tout va avec des `onClick`.
  - Fournir des attributs `aria-label` ou `sr-only` pour les boutons contenant uniquement des icônes.

## 4. Structure d'un Composant Shadcn/ui Standard

Chaque nouveau composant UI généré par l'agent doit respecter la structure suivante :

```
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@lib/utls";

const buttonVariants = cva(
 "inline-flex items-center justify-center whitespace-nowrap rounded-md text-sm font-medium transition-colors focus-
 {
 variants: {
 variant: {
 default: "bg-primary text-primary-foreground shadow hover:bg-primary/90",
 destructive: "bg-destructive text-destructive-foreground shadow-sm hover:bg-destructive/90",
 outline: "border border-input bg-background shadow-sm hover:bg-accent hover:text-accent-foreground",
 secondary: "bg-secondary text-secondary-foreground shadow-sm hover:bg-secondary/80",
 ghost: "hover:bg-accent hover:text-accent-foreground",
 link: "text-primary underline-offset-4 hover:underline",
 },
 size: {
 default: "h-9 px-4 py-2",
 sm: "h-8 rounded-md px-3 text-xs",
 lg: "h-10 rounded-md px-8",
 icon: "h-9 w-9",
 },
 },
 defaultVariants: {
 variant: "default",
 size: "default",
 },
 }
);

export interface ButtonProps
 extends React.ButtonHTMLAttributes<HTMLButtonElement>,
 VariantProps<typeof buttonVariants> {
 asChild?: boolean;
}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
 ({ className, variant, size, ...props }, ref) => {
 return (
 <button
 className={cn(buttonVariants({ variant, size, className }))}
 ref={ref}
 {...props}
 />
);
 }
);
Button.displayName = "Button";
```



```
export { Button, buttonVariants };
```

## 5. Checklist de Validation UI/UX

Avant de livrer un composant ou une page UI, l'agent doit valider les critères suivants :

- [ ] **Thématisation** : L'interface fonctionne-t-elle sans faille en Light mode ET Dark mode ?
- [ ] **Responsivité** : Le layout est-il parfaitement ajusté sur mobile (375px), tablette (768px) et desktop (1280px+) ?
- [ ] **Gestion des États (States)** : Les états `hover`, `focus`, `active`, `disabled` et `loading` (spinners/skeletons) sont-ils explicitement gérés ?
- [ ] **Skeletons & Placeholders** : Des composants `<Skeleton />` sont-ils affichés pendant le chargement des données ?
- [ ] **Lucide Icons** : Les icônes utilisées proviennent-elles de `lucide-react` avec des tailles cohérentes (h-4 w-4 ou h-5 w-5) ?
- [ ] **Accessibilité** : Les lecteurs d'écran peuvent-ils naviguer et lire les éléments interactifs ?

[http://googleusercontent.com/action\\_card\\_content/f3e62770-d29e-496b-a621-56fb47a3fcf0](http://googleusercontent.com/action_card_content/f3e62770-d29e-496b-a621-56fb47a3fcf0)

---